Optimizing the Database

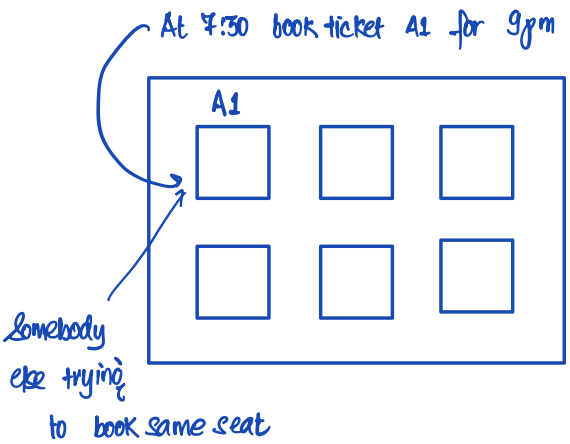
- 4 methods
  - → Concurrency control
  - → Clustering
  - → Partitioning
  - → Sharding

System Design Interview for Databases

Concurrency control → essential in RDBMS to ensure data integrity when updates occur to the database in a multi-user env.

Concurrency: when we try to perform 2 actions together or just one after other. Then it is known as Concurrent action or transactions

Concurrency Control means control these transactions.



→ Concurrency Control means control these transactions.

Two ways of Concurrency Control :→

(a) Lock Based Concurrency control

(b) Timestamp Based Concurrency Control

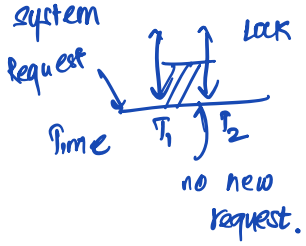
Lock-Based

We lock the transaction.

As soon as we get request, we lock entire system

→ until transaction is not terminated,

our database system will be under lock



{ Until first transaction is complete, second transaction won't start. }

A transaction can proceed only after concurrency control manager grants the lock to the transaction.

↓

Sometimes, both transactions keep on waiting for lock to end.

- It is called deadlock.

TimeStamp Based CC (Non-lock)

↓

Time of transactions

- T_1 ⟷ Timestamp (when it hits the server)
- T_2 ⟷ Timestamp

Until and unless the transaction with the smallest timestamp is processed. The second transaction doesn't start.

Queuing Mechanism based on Timestamp.

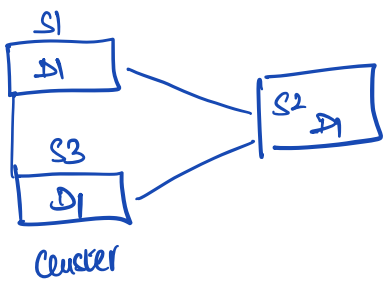
T_1 ⟷ 5:30 am ⟶ completes in 5 minutes & then T_2 starts.

T_2 ⟷ 5:32 am

Note :→ Concurrency control can occur only when multiple users have access to database.

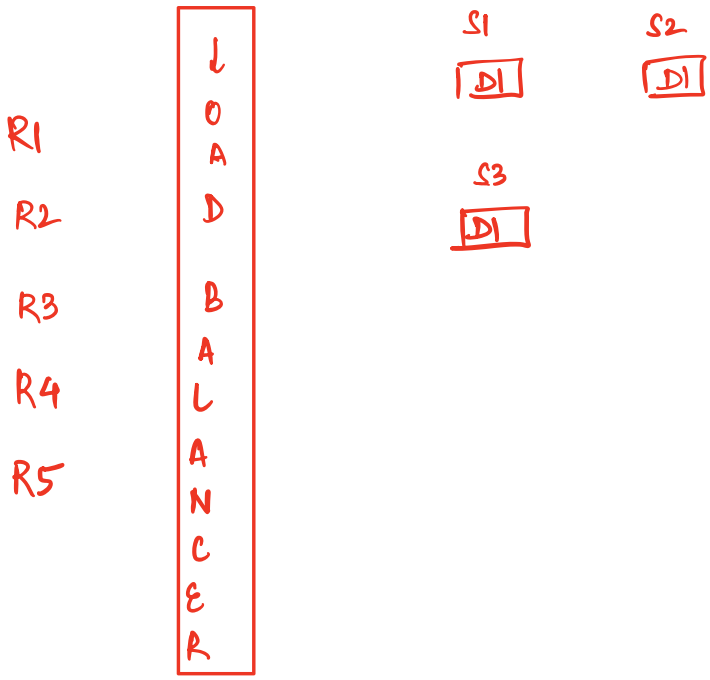
2-phase locking is a concurrency control method that ensures conflict serializability. The protocol utilizes locks, applied by a transaction to data, which may block other transactions from accessing the same data during the transaction's life. The timestamp-based concurrency control algorithm is a non-lock concurrency control method. This method also ensures conflict serializability. We know that deadlock is a situation where the transactions try to access lock on already locked data items. So, 2 phase locking may lead to deadlock states that can result in the mutual blocking of two or more transactions. But in timestamp, each transaction is allocated with the time slot. Hence it can't enter a deadlock. So Timestamp ordering ensures both the conflict serializability and is free from deadlocks

Clustering → uses clusters of database servers.



If same database is sitting on all servers – it is known as clustering

Advantages →
1. Redundancy
(Server 1 ← Hackers
So we shut down server 1 → now data doesn't exist anymore if only on server 1
2. Load Balancing
↓
eg 5 requests for same data
The logic of randomizing or sending the request to the correct server. (optimize the request in least time)
3. Availability



Use :→ By most websites using CDN (Content Delivery N/W)

↓

Eg If I am in Punjab

↓

A website which hosts video but its company - based in US

so, it'll take a lot of time.

To resolve this,

USA → India → Punjab

Availability ↑

Response Time ↓

Partitioning and Sharding

- Scaling
  - Vertical → Server → ↑ CPU Power, RAM } Very costly
  - Horizontal → Multiple servers
    - Vertical Partitioning
      - → slicing relation vertically
      - eg Emp_No and Name in Server 1
        Dept_ID and Address in Server 2
        Emp_Table
    - Horizontal Partitioning
      - ↳ slicing relation Horizontally.
      - eg 1-50,000 entries in S1
        50,000 – 1,00,000 → S2

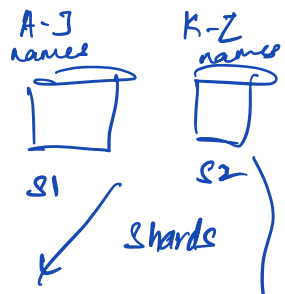
Emp_No	Name	Dept_ID	Address
4	Atul	Marketing	A-15

→ keep row & tuple intact

Sharding → extention of horizontal partitioning (sometimes used interchangeably)

facebook

User_ID	Profile	friends	Likes



They are mutually Independent

If request on S1 → S2 is free to get other request

Thus, we are scaling using sharding.

→ Shard can be further made smaller.

A distributed database is not limited to at least one system, it covers different sites, i.e, on multiple computers or over a network of computers. A distributed database system is found on various sites that don't share physical components. This might be required when a specific database must be accessed by various users globally. It must be managed in such a way that for the users it's like one single database.